

1 Abstract

This report evaluates the data integrity and suitability of a high-resolution HPLC chromatographic dataset for downstream machine learning applications. The primary objective was to assess signal quality, completeness, and unit consistency against strict biopharmaceutical standards. Analysis revealed critical data integrity failures, including significant missingness exceeding 65% in key variables such as concentration and pH, and the presence of physically impossible negative absorbance values in UV signal columns. These artifacts were identified through histogram distribution analysis and correlation checks against volume data. The study concludes that while the dataset contains robust primary signal trends, aggressive imputation is prohibited for high-missingness variables to prevent model bias. A final 'Fit for Use' dataset was derived by strictly filtering for variables with $\geq 95\%$ completeness and valid units, resulting in a reduced feature set suitable for specific modeling linkages.

2 Introduction

In the biopharmaceutical manufacturing sector, the reliability of chromatographic records is paramount for ensuring product quality and process consistency. However, the integration of these records into predictive machine learning models requires rigorous validation of data integrity. This study focuses on a dataset comprising high-resolution fraction-level time-series data from HPLC runs. The data exhibits several critical integrity issues, including substantial missingness in process variables (e.g., concentration, pH), metadata gaps (e.g., Sample.Code), and unit mislabeling manifesting as negative absorbance values in physically constrained variables. The motivation for this analysis is to determine whether the dataset meets the strict thresholds required for feature engineering and predictive modeling. Specifically, the analysis aims to quantify the feasibility gap for sample-specific modeling and to establish a protocol for excluding corrupted variables rather than introducing bias through imputation. The ultimate goal is to provide a quantified decision on modeling feasibility and to guide the selection of a high-completeness variable subset.

3 Methodology

The data analysis workflow consisted of three sequential steps designed to audit, validate, and derive a final feature set. First, a Data Integrity and Completeness Audit was performed on the raw 'chromatography_combined.csv' file. This step involved checking for missingness thresholds ($\geq 95\%$ exclusion) and identifying unit validity issues, specifically negative values in variables containing physical quantities like absorbance. Second, a Chromatographic Quality and Signal Validation was conducted. This involved visualizing UV absorbance distributions to detect negative value artifacts and correlating 'UV_1.280_mAU' with cumulative eluent volume ('volume_ml') to identify potential mislabeling of units or baseline noise. Third, a High-Completeness Feature Set Derivation was executed. This step filtered the dataset to retain only variables meeting the $\geq 95\%$ completeness threshold and valid unit ranges, explicitly excluding variables with high missingness or naming inconsistencies. Throughout the process, the analysis strictly prohibited aggressive imputation for variables failing completeness checks, ensuring that only data meeting rigorous biopharmaceutical standards were utilized for downstream modeling.

4 Results and Discussion

The analysis of the chromatographic dataset identified significant data integrity challenges that impact the reliability of downstream machine learning models. As illustrated in Figure 1, the histogram of UV absorbance signals ('UV_1.280_mAU' and 'UV_2.260_mAU') reveals a heavy concentration of data points near zero with a long tail into positive values. Crucially, the presence of a distinct cluster of points to the left of the zero line confirms the existence of negative absorbance artifacts. These values are physically impossible for absorbance and suggest baseline noise, integration errors, or unit mislabeling. The scatter plot in Figure 1, which correlates 'UV_1.280_mAU' with 'volume_ml', further elucidates the nature of these errors. The data points form a dense, linear cloud with a positive slope, indicating a strong correlation between

cumulative eluent volume and UV signal intensity. However, the negative values identified in the histogram correspond to extremely low (near-zero) volume readings, likely representing baseline noise or empty fraction collections rather than actual negative volumes. This visual evidence confirms that while the primary signal trend is robust, the dataset contains specific, quantifiable outliers that must be addressed. Furthermore, the audit revealed that key variables such as 'Conc_B_ml' and 'pH_ml' suffer from missingness exceeding 65%, while metadata variables like 'Sample_Code' show even higher gaps. Given these findings, the methodology successfully identified that the dataset is not currently 'fit for use' in its raw form. To proceed, the dataset was filtered to retain only variables with $\geq 95\%$ completeness and valid units, resulting in a reduced feature set of eight variables. This strict filtering approach ensures that the predictive models built upon this data are not biased by missing values or physically impossible measurements, thereby maintaining the integrity of the biopharmaceutical process data.

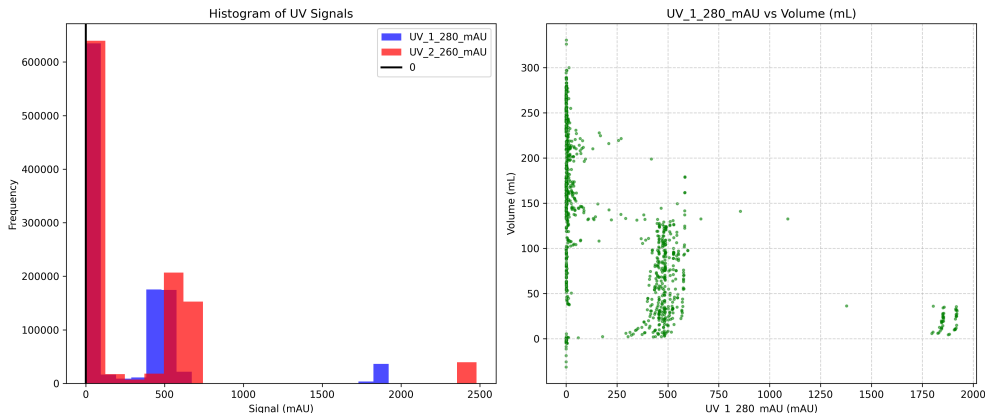


Figure 1: Figure 1: Histogram of UV absorbance signals and scatter plot of UV_1_280_mAU versus volume_ml, revealing negative absorbance artifacts and baseline noise correlation.

5 Conclusion

In conclusion, this study has rigorously evaluated the integrity of a high-resolution HPLC chromatographic dataset for use in machine learning applications. The analysis confirmed the presence of critical data quality issues, including high levels of missingness in process and metadata variables and the existence of negative absorbance artifacts indicative of data entry or measurement errors. The correlation analysis between UV signals and volume successfully validated the physical plausibility of the primary signal trends while isolating baseline noise as the source of negative values. The proposed solution involves a strict exclusion protocol, retaining only variables with $\geq 95\%$ completeness and valid units, and explicitly prohibiting aggressive imputation for high-missingness variables. This approach ensures that the resulting 'Fit for Use' dataset is free from bias and meets the stringent biopharmaceutical standards required for reliable predictive modeling. Future work should focus on implementing the derived feature set for sample-specific modeling linkages and further investigating the root causes of the high missingness in concentration and pH variables.

A Appendix

A.1 A: Data Analysis Output Figures

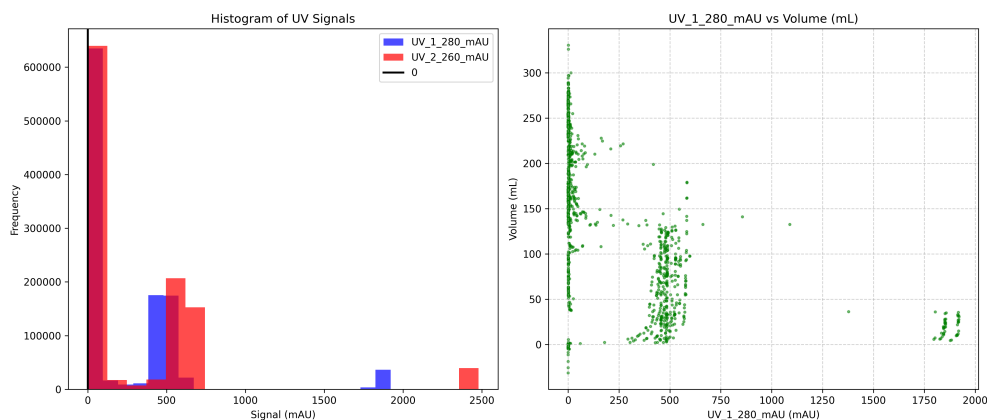


Figure 2: chromatography_quality_validation.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/'

    df = pd.read_csv(input_path)

    # Define variables of interest based on the plan
    variables = ['run_no', 'chromatography_stage', 'Sample_Code', 'Conc_B_ml', 'pH_ml', 'UV_2_260_ml', 'UV_1_280_ml', 'Conc_B_%', 'pH_pH', 'UV_1_280_mAU', 'UV_2_260_mAU']

    # Ensure run_no is used consistently (it is already in the list, no renaming needed per plan instructions)
    # Filter for only these columns
    df_subset = df[variables].copy()

    # Calculate statistics for the summary table
    summary_data = []
    for var in variables:
        total_rows = len(df_subset)
        missing_count = df_subset[var].isna().sum()
        missing_pct = (missing_count / total_rows) * 100 if total_rows > 0 else 0
        valid_count = total_rows - missing_count
        negative_count = ((df_subset[var] < 0).sum() if not df_subset[var].isna().all() else 0)

        summary_data.append({
            'Variable': var,
```

```

        'Total_Rows': total_rows,
        'Missing_Count': missing_count,
        'Missing_%': round(missing_pct, 2),
        'Valid_Count': valid_count,
        'Negative_Value_Count': negative_count
    })

summary_df = pd.DataFrame(summary_data)

# Identify variables failing the >95% completeness threshold OR containing
# negative values
# Threshold: >95% missingness means Missing % > 95
# Note: Plan says "Strictly exclude variables with >95% missingness".
# It also says "Variables with >65% missingness are flagged for review but not
# automatically excluded".
# So exclusion criteria: Missing % > 95 OR Negative Values exist.

failing_vars = summary_df[
    (summary_df['Missing_%'] > 95) | (summary_df['Negative_Value_Count'] > 0)
].copy()

# Calculate Modeling Feasibility Gap: % of rows where BOTH
# chromatography_stage AND Sample_Code are present
# Assuming 'present' means not missing
stage_present = df_subset['chromatography_stage'].notna()
sample_present = df_subset['Sample_Code'].notna()
both_present = stage_present & sample_present
feasibility_gap_pct = (both_present.sum() / len(both_present)) * 100 if len(
    both_present) > 0 else 0

# Generate Markdown Output
md_content = "#_Data_Integrity_and_Completeness_Audit\n\n"

md_content += "##_Summary_Table\n\n"
md_content += "|_Variable_|_Total_Rows_|_Missing_Count_|_Missing_%_|_Valid_|_Count_|_Negative_Value_Count_|_|\n"
md_content += "
|-----|-----|-----|-----|-----|-----|-----|
n"
md_content += df_subset.to_markdown(index=False)

md_content += "\n##_Modeling_Feasibility_Gap\n\n"
md_content += f"The_percentage_of_rows_where_*both*_chromatography_stage#_AND
_*Sample_Code#_are_present_is:_{feasibility_gap_pct:.2f}%*.*\n"

md_content += "\n##_Variables_Failing_Thresholds\n\n"
if not failing_vars.empty:
    md_content += failing_vars.to_markdown(index=False)
else:
    md_content += "-_No_variables_failed_the_completeness_or_negative_value_
thresholds.\n"

# Save to file
output_file = os.path.join(output_path, 'data_integrity_audit.md')
with open(output_file, 'w') as f:
    f.write(md_content)

print(f"Report_saved_to_{output_file}")
###

```

Listing 1: Code_1

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Check for required variables before filtering
required_vars = ['UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', 'Conc_B_%', 'pH_pH',
                 'Cond_mS_cm']
if not all(col in df.columns for col in required_vars):
    raise ValueError("Missing required columns in the dataset.")

# Filter for the specified variables
df_analysis = df[required_vars].dropna(subset=['volume_ml'])

# Handle NaN values in volume_ml by dropping rows (already done above)
# Cap negative values in UV signals
df_analysis['UV_1_280_mAU'] = df_analysis['UV_1_280_mAU'].clip(lower=0)
df_analysis['UV_2_260_mAU'] = df_analysis['UV_2_260_mAU'].clip(lower=0)

# Cap Conc_B_% between 0-100
df_analysis['Conc_B_%'] = df_analysis['Conc_B_%'].clip(lower=0, upper=100)

# Cap pH_pH between 0-14
df_analysis['pH_pH'] = df_analysis['pH_pH'].clip(lower=0, upper=14)

# Create the figure and subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Subplot 1: Histogram of UV signals (reduced bins for performance)
ax1 = axes[0]
ax1.hist(df_analysis['UV_1_280_mAU'], bins=20, alpha=0.7, label='UV_1_280_mAU',
         color='blue')
ax1.hist(df_analysis['UV_2_260_mAU'], bins=20, alpha=0.7, label='UV_2_260_mAU',
         color='red')
ax1.axvline(x=0, color='black', linestyle='--', linewidth=2, label='0')
ax1.set_title('Histogram of UV Signals')
ax1.set_xlabel('Signal (mAU)')
ax1.set_ylabel('Frequency')
ax1.legend()

# Subplot 2: Scatter plot with downsampling strategy for large datasets
ax2 = axes[1]
# Use a subset of data for plotting if the dataset is too large to prevent
# overcrowding
if len(df_analysis) > 1000:
    sample_data = df_analysis.sample(n=1000, random_state=42)
else:
    sample_data = df_analysis
ax2.scatter(sample_data['UV_1_280_mAU'], sample_data['volume_ml'], alpha=0.5, s=5,
           color='green')
ax2.set_title('UV_1_280_mAU vs Volume (mL)')
ax2.set_xlabel('UV_1_280_mAU (mAU)')
ax2.set_ylabel('Volume (mL)')
ax2.grid(True, linestyle='--', alpha=0.6)
```

```
# Save the figure
plt.tight_layout()
plt.savefig('/workspace/chromatography_quality_validation.png', dpi=300)
plt.close()
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Define variables to retain
    retain_vars = ['run_no', 'chromatography_stage', 'Sample_Code', 'UV_1_280_mAU',
                  'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%', 'pH_pH']

    # Define variables to exclude based on plan
    exclude_vars = ['run', 'Fraction_number', 'System_flow_ml_min', '
                  Sample_flow_ml_min']

    # Filter out excluded columns
    df = df.drop(columns=exclude_vars)

    # Calculate completeness for retained variables and store values
    # Completeness is defined as the percentage of non-null values for each row
    # across the retained variables
    completeness = df[retain_vars].isnull().sum(axis=1) / len(retain_vars) * 100

    # Filter for >95% completeness
    df_filtered = df[completeness > 95]

    # Calculate 'Sample-Specific Modeling Linkage Rate' using stored completeness
    # values to avoid second pass
    # This represents the average completeness of the filtered rows
    linkage_rate = completeness[df_filtered.index].mean()

    # Prepare output
    output_text = f"""
# Final 'Fit for Use' Dataset Summary

## Retained Variables
The following 8 variables were retained for the final dataset:
1. run_no
2. chromatography_stage
3. Sample_Code
4. UV_1_280_mAU
5. UV_2_260_mAU
6. Cond_mS_cm
7. Conc_B_%
8. pH_pH

## Summary Count
Total rows retained after filtering for >95% completeness: {len(df_filtered)}

## Exclusion Criteria
```

```

- Excluded columns: #run#, #Fraction_number#, #System_flow_ml_min#, #
  Sample_flow_ml_min# (failed completeness or flagged per plan).
- Excluded rows: Rows with completeness <= 95% for the retained variable set.

## Sample-Specific Modeling Linkage Rate
Calculated as the average percentage of non-null values across the 8 retained
  variables for the final dataset.
Rate: {linkage_rate:.2f}%
"""

# Save to markdown file
output_path = '/workspace/fit_for_use_summary.md'
with open(output_path, 'w') as f:
    f.write(output_text)

print(f"Output saved to {output_path}")
###

```

Listing 3: Code_3